What *is* a Compiler?

A function that maps an *input* string to an *output* string.

```
compiler :: String -> String
```

"Claude"
Text → Assembly | Bytecode | HTML
"program" "other language"
TEXT → STRUCTURED OUT

Typically, the *input* and *output* strings are "programs"

```
compiler :: SourceProgram -> TargetProgram
```

What *is* a Compiler?

For example, here are some well-known *compilers*

```
gcc, clang :: C          -> Binary          -- a.out, .exe
ghc        :: Haskell    -> Binary
javac      :: Java       -> JvmByteCode     -- .class
scalac     :: Scala      -> JvmByteCode
ocamlc     :: Ocaml      -> OcamlByteCode   -- .cmo
ocamlopt   :: Ocaml      -> Binary
gwt        :: Java       -> JavaScript      -- .js
v8         :: JavaScript -> Binary
nasm       :: X86        -> Binary
pdftex     :: LaTeX      -> PDF
pandoc     :: Markdown   -> PDF | Html | Doc
```

"meaning" → "meaning"

Key Requirements on output program:

1. Has the *same meaning* ("semantics") as input,
2. Is *executable* in relevant *context* (VM, microprocessor, web browser).

A Bit of History

FORTRAN → EXEC

Compilers were invented to [avoid writing machine code by hand][soap-fortran-assembly]

Richard Hamming – The Art of Doing Science and Engineering, p25:

> In the beginning we programmed in absolute binary… Finally, a Symbolic Assembly Program was devised – after more years than you are apt to believe during which most programmers continued their heroic absolute binary programming. At the time [the assembler] first appeared I would guess about 1% of the older programmers were interested in it – using [assembly] was "sissy stuff", and a real programmer would not stoop to wasting machine capacity to do the assembly.

John A.N. Lee, Dept of Computer Science, Virginia Polytechnical Institute

> One of von Neumann's students at Princeton recalled that graduate students were being used to hand assemble programs into binary for their early machine. This student took time out to build an assembler, but when von Neumann found out about it he was very angry, saying that it was a waste of a valuable scientific computing instrument to use it to do clerical work.

SRC "parser" → AST "TYPECHECK" → AST → IR → ASM → link → EXE
LIBRARY + RUNTIME

What does a Compiler *look like*?

An input source program is converted to an executable binary in many stages:

- **Parsed** into a data structure called an **Abstract Syntax Tree**
- **Checked** to make sure code is well-formed (and well-typed)
- **Simplified** into some convenient **Intermediate Representation**
- **Optimized** into (equivalent) but faster program
- **Generated** into assembly x86
- **Linked** against a run-time (usually written in C)

What is CSE 231 ?

- A *bridge* between two worlds
  - *High-level:* ML (**CSE 130/230**)
  - *Machine Code:* X86/ARM (**CSE 30**)

130 → 30
Human → Machine

A sequel to both those classes.

- How to write **a compiler** for Snek `->` `X86`
  1. Parsing
  2. Checking & Validation
  3. Simplification & Normalizing
  4. Optimization
  5. Code Generation
- But also, how to write **complex programs**
  1. Design
  2. Implement
  3. Test
  4. **Iterate**

How write a Compiler?

42

General recipe, applies to any large system

- *gradually, one feature at a time!*

We will

- **Step 1** Start with a teeny tiny language,
- **Step 2** Build a full compiler for it,
- **Step 3** Add a few features,
- **Go to** Step 2.

(Yes, loops forever, but we will hit Ctrl-C in 10 weeks…)

Policies: Lectures

N − 5

- We will not podcast lectures.
- 20% We will have worksheets to be filled in and submitted in every lecture.
  came out for tables(5
- We have a no-screens policy: students must keep their devices off during lectures.
- We require all exams be taken on the announced dates and times

Policies: Grading

Your grade will be calculated from assignments, exams and worksheets.

- (8-9) Assignments [30%] are given periodically, typically at one or two week intervals. On each you'll get a score from 0-3 (Incomplete/No Pass, Low Pass, Pass, High Pass).
- (2/3) Midterm Exams [50%] There are three exams in the course, one in week 5 and one in week 9, given in the Friday discussion sections, and one in the finals week. Your top two exams will be counted.
- (daily) Worksheets [20%] Every lecture will come with a 1-2 page handout, that must be filled in and submitted at the end of the lecture. Credit is given for reasonable effort in engaging with the notes from the day on the handout. Turn in 75% of the worksheets to get full credit.

Course Outline

We will **write a compiler** for `Snek -> X86`

So we will write *many* compilers: (add1 (add1 42))

- Numbers and increment/decrement
- Local Variables
- Nested Binary Operations +, −, *
- Booleans, Branches and Dynamic Types {int + bool}
- Functions
- Tuples and Structures ← Heap
- Lambdas and closures ← (\x → x + y)
- Types and Inference
- Garbage Collection
- Register Allocation

What will *you learn* ?

Core principles of compiler construction

- Managing Stacks & Heap "Calling convention"
- Type Checking
- Intermediate forms
- Optimization

Rust + C

Several new languages

- `Rust` to write the compiler
- C to write the "run-time"
- X86 compilation target

***More importantly* how to write a large program**

- How to use types for design
- How to add new features / refactor
- How to test & validate

What do you *need to know* ?

This 131 **depends very heavily** on CSE 130 / 230

- Datatypes (e.g. Lists, Trees, ADTs)
- Polymorphism
- Recursion

Also **depends on** CSE 30

- Experience with some C programming
- Experience with some assembly (x86)

Lets Begin!

Adder

42 →

A tiny language with 32-bit integers and three operations: `add1`, `sub1`, and `negate`.

Goals for the first assignment

- See how **all** compiler infrastructure fits together *end to end*
- Go from source text to a running binary
- Build each piece from scratch

The Big Picture

SRC

> "Compiling" an expression means generating assembly instructions that evaluate it and leave the answer in the `rax` register.

src → asm

The pipeline:

.s val
rax ← val

```
source file (.snek) ---------> x86 binary (.run)
```

.snek

The Runtime (`runtime/start.rs`)

SRC

A small Rust program that calls our compiled code and prints the result

```rust
#[link(name = "our_code")]
extern "C" {
    #[link_name = "\x01our_code_starts_here"]
    fn our_code_starts_here() -> i64;
}

fn main() {
  let i : i64 = unsafe {
    our_code_starts_here()
  };
  println!("{i}");
}
```

print

RUNTIME main() { our_code() } + our_code ASM

Key points

- `#[link(name = "our_code")]` — expects a precompiled `libour_code.a`
- `our_code_starts_here` — takes no args, returns a 64-bit integer
- `unsafe` — Rust can't verify the foreign function's signature; we're asserting it

our_code_starts_here:
mov rax, 42

Assembly Basics

A hand-written assembly file that returns 31

```
section .text
global our_code_starts_here
our_code_starts_here:
  mov rax, 31
  ret
```

- `mov rax, 31` — load the constant 31 into the return register
- `ret` — return to caller (result is whatever is in `rax`)

Assembling and Linking

On Linux:

```
$ nasm -f elf64 test/31.s -o runtime/our_code.o
$ ar rcs runtime/libour_code.a runtime/our_code.o
$ rustc -L runtime/ runtime/start.rs -o test/31.run
$ ./test/31.run
31
```

On MacOS:

```
$ nasm -f macho64 test/31.s -o runtime/our_code.o
$ ar rcs runtime/libour_code.a runtime/our_code.o
$ rustc --target x86_64-apple-darwin -L runtime/ runtime/start.rs -o
↪ test/31.run
$ ./test/31.run
31
```

Steps

1. `nasm` **assembles** `.s` → `.o` (machine code + label info)
2. `ar` **archives** `.o` → `.a` (standard library format)
3. `rustc` **links** `.a` + `start.rs` → executable

Generating Assembly from Rust

A compiler is just a function: `String → String`

```rust
fn compile(program: String) -> String {
    let num = program.trim().parse::<i32>().unwrap();
    format!("mov rax, {}", num)
}
```

Wrap it in a `main` that reads/writes files

```rust
fn main() -> std::io::Result<()> {
    let args: Vec<String> = env::args().collect();
    let in_name = &args[1];
    let out_name = &args[2];

    let mut in_file = File::open(in_name)?;
    let mut in_contents = String::new();
    in_file.read_to_string(&mut in_contents)?;

    let result = compile(in_contents);
    let asm_program = format!("
section .text
global our_code_starts_here
our_code_starts_here:
  {}
  ret
", result);

    let mut out_file = File::create(out_name)?;
    out_file.write_all(asm_program.as_bytes())?;
    Ok(())
}
```

Makefile

Automate the compile-assemble-link steps (use `elf64` on Linux, `macho64` on Mac)

```
test/%.s: test/%.snek src/main.rs
    cargo run -- $< test/$*.s

test/%.run: test/%.s runtime/start.rs
    nasm -f macho64 test/$*.s -o runtime/our_code.o
    ar rcs runtime/libour_code.a runtime/our_code.o
    rustc --target x86_64-apple-darwin -L runtime/ runtime/start.rs -o
↪ test/$*.run
```

Then just run

```
$ make test/37.run
```

49.snek compiler → 49.s nasm → 49.o → rustc → 49.run
runtime/start.rs

The Adder Language

Three pieces define any language we compile

1. **Concrete syntax** — the text the programmer writes
2. **Abstract syntax** — a Rust datatype our compiler uses
3. **Semantics** — the behavior (what values programs produce)

Concrete Syntax

(sub1 (add1 42))
(add1 (sub1 64))
79

```
<expr> :=
  | <number>
  | (add1 <expr>)
  | (sub1 <expr>)
  | ~~(negate <expr>)~~
```

Uses **s-expression** style — parenthesized prefix notation

- Easy to parse (many libraries available)
- No ambiguity, no precedence rules
- Same style as Scheme, Lisp, WebAssembly

Abstract Syntax

```rust
enum Expr {
    Num(i32),
    Add1(Box<Expr>),
    Sub1(Box<Expr>),
    Negate(Box<Expr>),
}
```

Why `Box<Expr>` and not just `Expr`?

- `Expr` is recursive — Rust needs to know the size at compile time
- `Box<T>` is a heap pointer — always a fixed size (64-bit address)
- Same role as a pointer in a C struct

Semantics

expr → i32

Adder programs always evaluate to a single `i32`

Expression	Result
`Num(n)`	`n`
`Add1(e)`	`eval(e) + 1`
`Sub1(e)`	`eval(e) - 1`
`Negate(e)`	`eval(e) * (-1)`

Examples

- `(add1 (sub1 5))` → 5
- `(negate (add1 3))` → -4

Interpreter (Warm-up)

```rust
fn eval(e: &Expr) -> i32 {
    match e {
        Expr::Num(n)       => *n,
        Expr::Add1(e1)     => eval(e1) + 1,
        Expr::Sub1(e1)     => eval(e1) - 1,
        Expr::Negate(e1)   => -eval(e1),
    }
}
```

(add1 (add1 42)) "parser" → AST "compiler" → ASM
./foo.run
44

Test with `cargo test`

```rust
#[cfg(test)]
mod tests {
    #[test]
    fn test1() {
        let expr1 = Expr::Num(10);
        assert_eq!(eval(&expr1), 10);
    }
}
```

(add1 (add1 49))
mov rax, 49
add1 rax, 1
add1 rax, 1

Parsing: S-Expressions

⟨e⟩ →
42
42
;; result of e
;; is in RAX

Add the `sexp` crate to `Cargo.toml`

```toml
[dependencies]
sexp = "1.1.4"
```

Parse a string into an `Expr`

```rust
use sexp::*;
use sexp::Atom::*;

fn parse_expr(s: &Sexp) -> Expr {
  match s {
    Sexp::Atom(I(n)) => Expr::Num(i32::try_from(*n).unwrap()),
    Sexp::List(vec) => {
      match &vec[..] {
        [Sexp::Atom(S(op)), e] if op == "add1"    =>
↪   Expr::Add1(Box::new(parse_expr(e))),
        [Sexp::Atom(S(op)), e] if op == "sub1"    =>
↪   Expr::Sub1(Box::new(parse_expr(e))),
        [Sexp::Atom(S(op)), e] if op == "negate"  =>
↪   Expr::Negate(Box::new(parse_expr(e))),
        _ => panic!("parse error"),
      }
    },
    _ => panic!("parse error"),
  }
}
```

- `Sexp::Atom(I(n))` — matches an integer literal
- `Sexp::List(vec)` — matches a parenthesized list
- Pattern guards (`if op == "add1"`) select which operation

Code Generation

```rust
fn compile_expr(e: &Expr) -> String {
    match e {
        Expr::Num(n)      => format!("mov rax, {}", *n),
        Expr::Add1(e1)    => compile_expr(e1) + "\nadd rax, 1",
        Expr::Sub1(e1)    => compile_expr(e1) + "\nsub rax, 1",
        Expr::Negate(e1)  => compile_expr(e1) + "\nneg rax",
    }
}
```

Key idea

- Recursively compile the subexpression — result lands in `rax`
- Then emit one instruction to transform `rax`

Example: `(sub1 (sub1 (add1 73)))`

```
mov rax, 73
add rax, 1
sub rax, 1
sub rax, 1
```

Result: 72

End-to-End Test

```
$ cat test/add.snek
(sub1 (sub1 (add1 73)))

$ make test/add.run
...

$ ./test/add.run
72
```

The four pieces that compose our compiler

1. **Assembler** (`nasm`) + **runtime** (`start.rs`) — build runnable binaries
2. **Abstract syntax** (`enum Expr`) — represent programs as data
3. **Parser** (`parse_expr`) — text → `Expr`
4. **Code generator** (`compile_expr`) — `Expr` → assembly string

These same four pieces appear in every assignment this quarter.

Your TODOs

1. Follow the tutorial and build the project from scratch
2. Add support for `negate` (parsing + code generation + tests)
3. Write at least 5 end-to-end tests; copy the terminal interaction into `transcript.txt`
4. Push your repository to the `01-adder` assignment on GitHub